ระบบบริหารจัดการงานแจ้งซ่อมไอทีตามกระบวนการ STATE MACHINE

Smart IT Helpdesk & Service Tracking System

ระบบจัดการและติดตามงานแจ้งซ่อมอุปกรณ์ไอทีแบบรวมศูนย์ ควบคุมสถานะด้วย State Machine พร้อมระบบส่งงานกลับแก้ไขเมื่อพบปัญหาซ้ำ (Pending Fix) และแดชบอร์ดสถิติ

1

คนที่ 1 : FRONTEND & BUSINESS FLOW

ผู้ดูแลส่วนติดต่อผู้ใช้, ขั้นตอนการแจ้งซ่อม และ Live Demo (นาที 0:00 - 4:30)

2

คนที่ 2 : BACKEND ARCHITECTURE & CORE LOGIC

ผู้ดูแลสถาปัตยกรรม Laravel, State Machine, ความปลอดภัย และ Dashboard (นาที 4:30 - 9:00)

ภาพรวมทีมงาน

การแบ่งหน้าที่และความรับผิดชอบของสมาชิก (2 คน)

คนที่ 1 (Speaker 1) — เวลาพูด: ~4.5 นาที

Frontend, UI/UX & กระบวนการผู้ใช้งาน

รับผิดชอบส่วนติดต่อผู้ใช้ และการโต้ตอบตามวงจรชีวิตงานแจ้งซ่อม

- ✓ **UI Components (Blade + Tailwind CSS):** ออกแบบฟอร์มแจ้งซ่อม รายการตาราง ค้นหา และการ์ดแสดงผลสถานะ
- ✓ **State Machine Interactive Box:** ส่วนคำสั่งที่เปลี่ยนรูปแบบตามสิทธิ์และสถานะงานแบบไดนามิก
- ✓ **Rework Interaction Flow:** รองรับการส่งกลับแก้ไขเมื่อพบปัญหาซ้ำ และการกดยืนยันซ่อมเสร็จของช่าง
- ✓ **Live Demonstration:** สาธิตการทดลองใช้งานระบบจริงหน้าห้อง

เทคโนโลยี: Blade Templates, Tailwind CSS, Alpine.js

คนที่ 2 (Speaker 2) — เวลาพูด: ~4.5 นาที

Backend Architecture & Core Logic

รับผิดชอบสถาปัตยกรรม Laravel, ตรรกะธุรกิจ, ความปลอดภัย และฐานข้อมูล

- ✓ **System Architecture (MVC + Service):** ออกแบบ `TicketStatusService` แยก Business Logic ออกจาก Controller
- ✓ **State Machine & Transactions:** ควบคุมกฎการเปลี่ยนสถานะ และความสมบูรณ์ข้อมูลด้วย `DB::transaction()`
- ✓ **Security & RBAC:** ควบคุมสิทธิ์ด้วย `RoleMiddleware` และ PHP 8.2 Backed Enums
- ✓ **Dashboard Analytics & KPIs:** คำนวณระยะเวลาเฉลี่ย (julianday), CSAT Rating และสถิติรายช่าง

เทคโนโลยี: Laravel 11, Eloquent ORM, SQLite/MySQL, Backed Enums

คนที่ 1 นำเสนอ

ที่มา ปัญหา และวัตถุประสงค์ของโครงงาน

✕ ปัญหาของระบบแจ้งซ่อมแบบเดิม (Pain Points)

1. การแจ้งซ่อมไร้ระบบบันทึก (LINE / วาจา)

ข้อมูลตกหล่น ไม่ทราบผู้รับผิดชอบ ไม่มีประวัติย้อนหลังสำหรับการตรวจสอบ

2. ปัญหาซ้ำ (Repeat Issues) ไม่ถูกจัดการ

เมื่อซ่อมเสร็จแต่อาการเดิมยังไม่หาย ผู้แจ้งต้องเปิดใบงานใหม่ซ้ำซ้อน เสียเวลาทั้งสองฝ่าย

3. ขาดตัวชี้วัดประสิทธิภาพ (KPIs)

หัวหน้างานไม่สามารถวัดระยะเวลาเฉลี่ยในการซ่อม และคะแนนความพึงพอใจของผู้ใช้ได้

✓ แนวทางแก้ไขและวัตถุประสงค์ (Objectives)

1. รวมศูนย์ข้อมูลบนเว็บแอปพลิเคชัน (Single Source)

บันทึกรายละเอียด อาการเสีย หมวดหมู่อุปกรณ์ ระดับความสำคัญ และรูปภาพแนบ

2. กระบวนการส่งกลับไปแก้ไข (Rework Workflow)

รองรับสถานะ "รอดำเนินการแก้ไข" ให้ช่างซ่อมซ้ำและกดยืนยันเสร็จได้โดยไม่ต้องเปิดเคสใหม่

3. ตรวจสอบย้อนหลังและประเมินผลอย่างโปร่งใส

มีระบบ Audit Trail บันทึกทุกขั้นตอน และให้คะแนน 1-5 ดาวก่อนปิดงานจริง

บทบาทผู้ใช้งาน และวงจรสถานะงาน (State Lifecycle)

ROLE 1 : USER

ผู้แจ้งซ่อม (พนักงานทั่วไป)

เปิด Ticket แจ้งปัญหา, ติดตามสถานะ, สนทนาตอบโต้, ยืนยันผลการซ่อม/ส่งกลับแก้ไข และให้คะแนนดาวความพึงพอใจ

ROLE 2 : TECHNICIAN

ช่างเทคนิค (IT Support)

รับงานที่ได้รับมอบหมาย, เริ่มงานซ่อม, ดำเนินการแก้ไขปัญหาซ้ำ, แนบรูปหลักฐานผลซ่อม, กดยืนยันซ่อมเสร็จ

ROLE 3 : ADMIN

ผู้ดูแลระบบ (IT Manager)

คัดกรองงาน, มอบหมายงานให้ช่างที่เหมาะสม, จัดการหมวดหมู่/ผู้ใช้ และติดตามสถิติภาพรวมผ่านแดชบอร์ด

วงจรกระบวนการ 6 สถานะหลัก (LIFECYCLE PIPELINE)

1. OPEN	2. ASSIGNED	3. IN PROGRESS	4. RESOLVED	5. CLOSED
รอรับเรื่อง	มอบหมายช่าง	กำลังตรวจซ่อม	ซ่อมเสร็จแล้ว	ปิดงาน + ประเมิน

↩ **กรณีพบปัญหาซ้ำ (จาก Resolved):** ผู้แจ้งกดส่งกลับแก้ไข → เปลี่ยนสถานะเป็น **Pending Fix (รอดำเนินการแก้ไข)** → ช่างตรวจสอบแก้ไข → กดซ่อมเสร็จ → วนกลับไป Resolved

ฟังก์ชันเด่น: การจัดการปัญหาซ้ำ (Rework Workflow)

การกระทำของผู้แจ้ง (REQUESTER) — ณ สถานะ Resolved

ปุ่ม "พบปัญหาซ้ำ / ส่งกลับไปแก้ไข"

เมื่อช่างแจ้งว่าซ่อมเสร็จแล้ว แต่ผู้แจ้งตรวจสอบพบว่าอุปกรณ์ยังมีอาการเดิม สามารถเลือกตัวเลือกส่งกลับแก้ไข พร้อมพิมพ์ระบุอาการที่พบ

• ผลลัพธ์ในระบบ:

1. ปรับสถานะเป็น **รอดำเนินการแก้ไข (Pending Fix)**
2. จัดกลุ่มอยู่ภายใต้ **In Progress** ในตัวกรองและแดชบอร์ด
3. บันทึกประวัติลง Audit Logs, ข้อความสนทนา และแจ้งเตือนผ่าน LINE

การกระทำของช่าง (TECHNICIAN) — ณ สถานะ PendingFix

ปุ่ม "กดซ่อมเสร็จแล้ว" สำหรับช่าง

ช่างเทคนิคจะเห็นกล่องแจ้งเตือนสีแดงพร้อมเหตุผลที่ถูกส่งกลับ เมื่อเข้าตรวจสอบแก้ไขเพิ่มเติมเสร็จสิ้นแล้ว สามารถกดยืนยันงานเสร็จได้ทันที

• ผลลัพธ์ในระบบ:

1. ช่างบันทึกผลการแก้ไขซ้ำ และสามารถแนบรูปถ่ายหลักฐาน
2. กดปุ่ม `[ ✓ กดซ่อมเสร็จแล้ว ]` เพื่อปรับสถานะกลับเป็น **Resolved**
3. ส่งต่อให้ผู้แจ้งทดสอบใช้งานและประเมินดาวเพื่อปิดงานสมบูรณ์

คนที่ 2 นำเสนอ

สถาปัตยกรรมระบบด้วย Laravel 11 Framework

04:30 - 05:30

เสาหลักที่ 1

MVC + Service Pattern

แยกตรรกะธุรกิจที่ซับซ้อนออกจาก Controller โดยใช้ `TicketStatusService` ทำให้โค้ดสะอาด ดูแลรักษาง่าย และนำไป Reuse ได้

```
Controller → Service → Model / DB
```

เสาหลักที่ 2

PHP 8.2 Backed Enums

ใช้ Enums กำหนดค่าคงที่: `TicketStatus`, `TicketPriority`, `UserRole` พร้อม Method แสดง Label และสี Badge ในตัว

```
TicketStatus::PendingFix→label()
```

เสาหลักที่ 3

Eloquent ORM Model

เชื่อมโยงตารางข้อมูลผ่าน Eloquent: Ticket สัมพันธ์กับ Requester, Technician, Category, Comments, StatusLogs และ Rating อย่างเป็นระบบ

```
HasMany / BelongsTo / HasOne
```

โครงสร้างไฟล์หลัก: `app/Services/TicketStatusService.php`, `app/Enums/TicketStatus.php`, `app/Http/Controllers/TicketController.php`

กลไก State Machine และความถูกต้องของข้อมูล (Data Integrity)

การควบคุมกฎการเปลี่ยนสถานะ (State Machine Rules)

ใน `TicketStatusService::validateTransition()` มีการจำกัดเส้นทางการเปลี่ยนสถานะเพื่อป้องกันการข้ามขั้นตอนโดยไม่ได้รับอนุญาต

```
// กฎการเปลี่ยนสถานะที่ถูกอนุญาต
Open → Assigned
Assigned → InProgress
InProgress → Resolved
Resolved → Closed | PendingFix
PendingFix → Resolved
```

Atomic DB Transaction & Audit Trail

ทุกการเปลี่ยนสถานะถูกครอบด้วย `DB::transaction()` เพื่อให้การอัปเดตเกิดขึ้นพร้อมกันครบทุกส่วน หากมีจุดใดล้มเหลว ระบบจะ Rollback คืนค่าเดิมทันที

1. อัปเดตตาราง `tickets` (สถานะ, วันที่เสร็จ/ปิดงาน)
2. บันทึก Audit Log ลงตาราง `status_logs` อัตโนมัติ
3. สร้างประวัติสนทนาลงในตาราง `comments`
4. ส่งข้อความแจ้งเตือนผ่าน `LineMessagingService`

คนที่ 2 นำเสนอ

ความปลอดภัยและการควบคุมสิทธิ์ (RBAC Security)

ชั้นที่ 1 Route-Level Middleware

ใช้ `RoleMiddleware` ป้องกันการเข้าถึง URL ในระดับ Routing เช่น เส้นทาง `/admin/*` ป้องกันไม่ให้ User ทั่วไปเข้าถึง

```
Route::middleware(['auth', 'role:Admin'])→prefix('admin')
    →group(function () { ... });
```

ชั้นที่ 2 Action-Level Authorization

ตรวจสอบสิทธิ์ในฟังก์ชัน `checkPermission()` ละเอียดถึงระดับบุคคลและสถานะงาน:

- **มอบหมายงาน:** เฉพาะ Admin เท่านั้น
- **กดซ่อมเสร็จ:** เฉพาะช่างเทคนิค (Technician) หรือ Admin
- **ยืนยันปิดงาน / ส่งกลับแก้ไข:** เฉพาะเจ้าของใบแจ้งซ่อม (Ticket Owner)

คนที่ 2 นำเสนอ

แดชบอร์ดสถิติและการวิเคราะห์ผล (Admin BI Dashboard)

ใบแจ้งซ่อมทั้งหมด

Total Tickets

ติดตามแบบ Real-Time

กำลังดำเนินการ

In Progress

นับรวมงานรอแก้ไขซ้ำ

เวลาเฉลี่ยในการซ่อม

Avg. Resolution

คำนวณผ่าน SQLite/MySQL

ความพึงพอใจเฉลี่ย

CSAT Rating

คะแนนเฉลี่ย 1 - 5 ดาว

การจัดกลุ่มสถานะ (Status Grouping Analysis)

สถานะ **"รอดำเนินการแก้ไข (PendingFix)"** ถูกจัดให้อยู่ในกลุ่ม **In Progress** เพื่อให้ยอดรวมงานที่กำลังซ่อมแซมสะท้อนปริมาณงานจริงของฝ่าย IT อย่างถูกต้อง

ประสิทธิภาพรายช่าง (Technician Performance)

ระบบคำนวณยอดงานที่ได้รับมอบหมาย เทียบกับยอดงานที่ซ่อมเสร็จสิ้นรายบุคคล ช่วยให้หัวหน้าทีม IT วางแผนกระจายงานได้อย่างเท่าเทียมและมีประสิทธิภาพ

ทั้ง 2 คนร่วมสรุป

สรุปผลลัพธ์โครงงาน และช่วงถาม-ตอบ (Q&A)

09:00 - 10:00

สไลด์ 10 / 10

01

ระบบทำงานได้สมบูรณ์จริง

ครอบคลุมวงจรชีวิตงานซ่อมตั้งแต่แจ้งซ่อม มอบหมาย ซ่อมแซม จัดการปัญหาซ้ำ จนถึงการประเมินดาวปิดงาน

02

Laravel Architecture แข็งแกร่ง

ประยุกต์ใช้ Service Pattern, DB Transactions, Eloquent ORM และ Enums ตามมาตรฐานซอฟต์แวร์ระดับสากล

03

การแบ่งงาน 2 คนที่มีประสิทธิภาพ

แยกส่วน Frontend/UX และ Backend Engine ทำให้สามารถพัฒนาและต่อยอดระบบได้อย่างรวดเร็วและเป็นมืออาชีพ

ขอขอบคุณคณะกรรมการและอาจารย์ทุกท่าน

ยินดีรับฟังคำถาม ข้อเสนอแนะ และพร้อมสาธิตซอร์สโค้ดเพิ่มเติมครับ